

ANGULAR LIFECYCLE HOOKS

LET'S UNDERSTAND



Written by
Petros Sargsian



WHAT ARE LIFECYCLE HOOKS?

Lifecycle hooks are methods Angular calls at specific moments during a component's life.

- Component **creation** to **destruction**
- Called by Angular **automatically**
- Implement **interfaces** to use

```
export class MyComponent implements OnInit, OnDestroy {  
  constructor() {  
    console.log('1. Constructor called');  
  }  
  ngOnInit() {  
    console.log('2. ngOnInit called');  
  }  
  ngOnDestroy() {  
    console.log('3. ngOnDestroy called');  
  }  
  // and more  
}
```



Written by
Petros Sargsian



LIFECYCLE HOOK ORDER

Execution Order:

1. constructor
2. ngOnChanges (if @Input)
3. ngOnInit
4. ngDoCheck
5. ngAfterContentInit
6. ngAfterContentChecked
7. ngAfterViewInit
8. ngAfterViewChecked
9. ngOnDestroy



Written by
Petros Sargsian



CONSTRUCTOR

TypeScript class **constructor** called when Angular instantiates the component. Runs **BEFORE** Angular initialization.

Use for: Dependency injection only

Avoid: Complex logic, DOM access, @Input access

```
export class UserComponent {  
  
    constructor(  
        private userService: UserService,  
        private router: Router  
    ) {  
        // GOOD: Inject dependencies  
  
        // BAD: Don't do this  
        // this.userService.getUsers();  
        // Component not ready yet!  
    }  
}
```



Written by
Petros Sargsian



NG ON CHANGES

Called when **@Input properties change**.
Receives SimpleChanges object
showing previous and current values.

Runs: Before ngOnInit and whenever
@Input changes

Use for: Reacting to @Input property
changes

```
export class ChildComponent implements OnChanges {
  @Input() user: User;

  ngOnChanges(changes: SimpleChanges) {
    if (changes['user']) {
      console.log('Previous:', changes['user'].previousValue);
      console.log('Current:', changes['user'].currentValue);

      // React to user change
      this.loadUserDetails();
    }
  }
}
```



Written by
Petros Sargsian



NG ON INIT

Called once **after first ngOnChanges**.
Component is initialized and @Input
properties are set.

Use for: Initial data loading, setup logic,
API calls

Most common hook for initialization

```
export class ProductComponent implements OnInit {  
  @Input() productId: number;  
  product: Product;  
  
  constructor(private service: ProductService) {}  
  
  ngOnInit() {  
    // GOOD: Component ready, @Input available  
    this.loadProduct();  
    this.setupForm();  
  
    // Access @Input safely  
    console.log('Product ID:', this.productId);  
  }  
  
  loadProduct() {  
    this.service.getProduct(this.productId)  
      .subscribe(p => this.product = p);  
  }  
}
```



Written by
Petros Sargsian



NG DO CHECK

Called during **every change detection cycle**. Use for custom change detection logic Angular can't detect.

Warning: Runs very frequently, keep it lightweight

Use for: Detecting changes Angular misses

```
export class CustomCheckComponent implements DoCheck {  
  @Input() items: string[];  
  private previousLength = 0;  
  
  ngDoCheck() {  
    // Detect array length change  
    if (this.items.length !== this.previousLength) {  
      console.log('Items array changed');  
      this.previousLength = this.items.length;  
      this.recalculate();  
    }  
  }  
}
```



Written by
Petros Sargsian



NG AFTER CONTENTINIT

Called once **after Angular projects external content** into component via `<ng-content>`. Content children are available.

Use for: Accessing `@ContentChild` elements

Runs once after content projection

```
export class TabsComponent implements AfterContentInit {
  @ContentChildren(TabComponent)
  tabs: QueryList<TabComponent>;

  ngAfterContentInit() {
    // Content projected, tabs available
    console.log('Tabs count:', this.tabs.length);

    // Select first tab
    if (this.tabs.first) {
      this.tabs.first.active = true;
    }
  }
}
```



Written by
Petros Sargsian



NG AFTER VIEW INIT

Called once **after component's view and child views are initialized**. View children are available.

Use for: DOM manipulation, accessing @ViewChild elements

Most common for interacting with DOM

```
export class ChartComponent implements AfterViewInit {
  @ViewChild('canvas')
  canvas: ElementRef<HTMLCanvasElement>;

  ngAfterViewInit() {
    // DOM ready, canvas available
    const ctx = this.canvas.nativeElement
      .getContext('2d');
    this.drawChart(ctx);

    // Initialize third-party libraries
    this.initializeChart();
  }
}
```



Written by
Petros Sargsian



NG AFTER CONTENT CHECKED

Called after every check of projected content. Runs after `ngAfterContentInit` and during every change detection.

Warning: Runs frequently, performance cost

Use for: Responding to content changes

```
export class AccordionComponent
  implements AfterContentChecked {

  @ContentChildren(PanelComponent)
  panels: QueryList<PanelComponent>;

  ngAfterContentChecked() {
    // Check if content changed
    // Keep lightweight!
    this.updatePanelStates();
  }
}
```



Written by
Petros Sargsian



NG AFTER VIEW CHECKED

Called after **every check of component's view and child views**. Runs after `ngAfterViewInit` and every change detection.

Warning: Runs very frequently, be careful
Use for: Responding to view changes

```
export class ScrollComponent implements AfterViewChecked {  
  @ViewChild('scrollContainer')  
  container: ElementRef;  
  
  ngAfterViewChecked() {  
    // Run after view updates  
    // CAUTION: Performance sensitive  
    this.adjustScrollPosition();  
  }  
}
```



Written by
Petros Sargsian



NG ON DESTROY

Called once **before Angular destroys the component**. Last chance for cleanup.

Use for: Unsubscribe, clear timers, detach listeners

Critical for: Preventing memory leaks

```
export class DataComponent implements OnDestroy {
  private destroy$ = new Subject<void>();
  private intervalId: any;

  ngOnInit() {
    this.service.data$
      .pipe(takeUntil(this.destroy$))
      .subscribe();

    this.intervalId = setInterval(() => {}, 1000);
  }

  ngOnDestroy() {
    // Cleanup subscriptions
    this.destroy$.next();
    this.destroy$.complete();

    // Clear timers
    clearInterval(this.intervalId);
  }
}
```



Written by
Petros Sargsian



CONSTRUCTOR VS NGONINIT

Constructor:

- Dependency injection only
- Component not ready
- @Input not available

ngOnInit:

- Initialization logic
- Component ready
- @Input available
- API calls



Written by
Petros Sargsian



COMMON USE CASES

ngOnInit:

- Load data, setup forms, initialize

ngOnChanges:

- React to @Input changes

ngAfterViewInit:

- DOM manipulation, third-party
libs

ngOnDestroy:

- Cleanup, unsubscribe, prevent
leaks



Written by
Petros Sargsian



BEST PRACTICES

Practices:

- Keep constructor simple, only DI
- Use `ngOnInit` for most initialization
- Always implement `ngOnDestroy` for cleanup
- Avoid logic in `ngDoCheck` and `AfterChecked`, performance
- Access DOM only in `AfterViewInit`



Written by
Petros Sargsian





VOILA!

Follow for more content like this.

↔ Repost

+ Follow me